1. 多线程的基本概念

1.1 线程的概念

线程是操作系统能够进行运算调度的最小单位，它被包含在进程之中，是进程内的实际执行单元。**进程是资源分配的最小单位，而线程是调度的最小单位。**与进程的创建不同，线程在创建时除了会申请线程本身所必须的资源之外，其他的资源比如内存空间、文件描述符等资源会使用进程中已经存在的资源，并且一个进程中的多个线程会共享本进程中的内存空间、文件描述符等大部分资源。需要注意的是，线程不具备进程级别的隔离性，一旦所属进程崩溃，虽然不会影响到其他进程，但该进程内的所有线程都将终止。

1.2 多线程与多进程的区别

在Python中，多线程和多进程都是实现多并发执行的技术，但它们在多个方面存在显著的区别：

1.2.1 调度

- **多线程（Threading）**：线程是操作系统能够进行运算调度的最小单位，被包含在进程之中，是进程中的实际运作单位。由于线程共享进程的内存空间，操作系统可以在同一进程空间中快速切换不同线程，实现并发执行。
- **多进程（Multiprocessing）**：进程是操作系统分配资源的最小单位。每个进程都有自己的内存空间，因此进程间的通信比线程要复杂，但在python中，多进程可以利用多核处理器的多个核心，实现真正的并行计算。

1.2.2 GIL（全局解释器锁）

- **多线程**：CPython解释器（Python的主要实现）有一个全局解释器锁（GIL），它确保同一时刻只有一个线程执行Python字节码。因此，即使在多核CPU上，使用多线程的Python程序也无法实现真正的并行计算，GIL限制了线程在执行CPU密集型任务时的效率。
- **多进程**：每个Python进程都有自己的Python解释器和内存空间，因此GIL不会限制多进程。多进程可以绕过GIL，充分利用多核处理器进行并行计算。

GIL锁的优点：

华清远见元宇宙实验中心 www.hqyj.com.cn

华清远见 HQYJ.COM

1. **简化内存管理**：由于GIL确保同一时刻只有一个线程在执行，因此CPython的内存管理可以设计得更加简单。它不需要考虑多个线程同时修改对象的情况，从而避免了复杂的多线程内存回收问题。
2. **易于实现**：GIL简化了CPython解释器的实现，因为它不需要考虑多线程并发执行时的数据竞争和同步问题。
3. **单线程性能**：在没有多线程竞争的情况下，单个线程的性能可以保持得很好，因为GIL避免了不必要的上下文切换。

GIL锁的缺点：

1. **限制了并发性能**：在多核处理器上，GIL使得即使在多线程程序中，也无法实现真正的并行计算。因为GIL确保在任何给定时间只有一个线程在执行，所以在CPU密集型任务中，多线程并不会带来性能上的提升。
2. **线程间竞争**：在多线程环境中，当线程试图获取GIL时，可能会发生频繁的上下文切换，这会导致性能下降，尤其是在线程数量较多时。
3. **不利于多核处理器**：随着多核处理器的普及，GIL成为了Python程序利用多核优势的一个障碍。尽管可以通过多进程来绕过GIL的限制，但这增加了程序的复杂性和资源消耗。
4. **需要额外的同步机制**：尽管GIL简化了单线程的内存管理，但在多线程程序中，为了防止数据竞争，开发者仍然需要使用锁或其他同步机制来保护共享资源，这增加了编程的复杂性。

1.2.3 内存共享

- **多线程**：由于线程共享内存，它们可以直接读写同一内存地址的数据，这使得数据共享变得简单，但同时也引入了线程安全问题。
- **多进程**：进程间内存是隔离的，数据共享需要通过进程间通信（IPC）机制，如管道、消息队列、共享内存等。

1.2.4 创建和销毁开销

- **多线程**：线程通常比进程轻量，创建和销毁线程的开销相对较小。
- **多进程**：进程比线程重，创建和销毁进程的开销相对较大。

1.2.5 适用场景

- **多线程**：适合于I/O密集型任务，如网络请求、文件读写等，因为这些任务常常需要等待，而GIL在等待期间会释放，使得其他线程可以执行。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **多进程**：适合于CPU密集型任务，如科学计算、图像处理等，因为它们可以并行运行而不会受到GIL的限制。

1.2.6 易用性

- **多线程**：由于共享内存和数据结构，多线程编程通常更简单，但也更容易出错。
- **多进程**：需要更多的代码来处理进程间通信和数据同步，但相对来说更安全。

2. Python中的多线程

在Python中，多线程是通过`threading`模块实现的，该模块允许程序员创建、启动、同步多个线程，并提供了一系列的API来支持线程间同步和通信。

在Python中，使用threading模块中的Thread类来创建线程，Thread类创建对象时的参数为：

```
threading.Thread(group=None, target=None, name=None, args=(),
kwargs=None, *, daemon=None)
```

以下是各个参数的说明：

- **group**: 应该始终为`None`，保留供未来扩展使用。
- **target**: 是一个可调用的对象（函数），该线程启动时，这个对象将被调用。如果不提供，则不会运行任何东西。
- **name**: 线程名称。默认情况下，将分配一个唯一的名称。
- **args**: 传递给`target`函数的位置参数，默认是一个元组。
- **kwargs**: 传递给`target`函数的关键字参数，默认是一个字典。
- **daemon**: 指定线程是否为守护线程。如果设置为`True`，则该线程会在主线程结束时自动退出。如果是None，则继承自创建它的线程。

以下是一些`Thread`类提供的主要方法和属性：

- **start()**：启动线程活动。使用start去启动线程，会调用run()方法，它会创建一个新的线程来执行run()方法中的代码。
- **run()**：表示线程活动的方法。可以在子类中重写此方法，重写之后执行重写的代码。通常不需要直接调用run方法，应该调用start方法去启动线程，如果直接调用run方法(没有通过start方法去启动线程)那么run方法中的代码将在当前线程中同步执行，而不是在新的线程中执行。
- **join(timeout=None)**：等待线程终止。`timeout`参数是可选的，表示等待的最长时间（以秒为单位）。如果没有指定`timeout`，则该方法将无限期等待。

华清远见嵌入式实验中心 www.lab.com.cn

华清远见 HQYJ.COM

- **is_alive()**：返回线程是否还活着。
- **getName()**：返回线程名。
- **setName(name)**：设置线程名。
- **isDaemon()**：返回线程的守护状态。
- **setDaemon(daemonic)**：设置线程的守护状态。必须在start开始前设置。
- **name**：线程名称。
- **ident**：线程的标识符。如果线程尚未启动，则为`None`。
- **daemon**：线程的守护状态。

2.1 创建不传参的线程

```
import time
from threading import Thread


def func1():
    print("Function 1 is running.")
    time.sleep(2)
    print("Function 1 has finished.")


def func2():
    print("Function 2 is running.")
    time.sleep(2)
    print("Function 2 has finished.")


if __name__ == '__main__':
    start_time = time.time()  # 记录程序开始时间

    # 创建一个线程来运行func2
    process = Thread(target=func2)
    process.start()

    # 主程序运行func1
    func1()

    # 等待线程结束
    process.join()

    # 计算并打印整个程序的运行时间
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    total_time = time.time() - start_time
    print(f"Total execution time: {total_time:.2f} seconds")
```

2.2 创建传参的线程

2.2.1 使用args传参

```
from threading import Thread
import time

def say_hello(name):
    time.sleep(2)
    print(f'Hello {name}, Nice to meet you!')

if __name__ == '__main__':
    start = time.time()
    thread1 = Thread(target=say_hello, args=('Alice', ))
    thread1.start()
    say_hello('Bob')
    thread1.join()
    exe_time = time.time() - start
    print(exe_time)
```

2.2.2 使用kwargs传参

```
from threading import Thread
import time

def say_hello(name):
    time.sleep(2)
    print(f'Hello {name}, Nice to meet you!')

if __name__ == '__main__':
    start = time.time()
    thread1 = Thread(target=say_hello, kwargs={'name': 'Alice'})
    thread1.start()
    say_hello('Bob')
    thread1.join()
    exe_time = time.time() - start
    print(exe_time)
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

2.3 Timer

Timer，也被称为定时器，它允许你在一定时间后执行一个函数或者可调用的对象。`Timer`类是`Thread`类的一个子类，因此它具有线程的所有特性，并且可以用来在后台执行定时任务。

以下是`Timer`类的基本用法和特性：

```
class threading.Timer(interval, function, args=None, kwargs=None)
```

2.3.1 参数说明

- **interval**: 一个浮点数或整数，表示在执行`function`之前需要等待的时间（以秒为单位）。
- **function**: 一个可调用的对象，当定时器到期时将被执行。
- **args**: 传递给`function`的位置参数元组。
- **kwargs**: 传递给`function`的关键字参数字典。

2.3.2 实例方法

- **start()**: 启动定时器。`Timer`将在指定的时间间隔后开始执行目标函数。
- **cancel()**: 取消定时器。如果定时器尚未启动，则此方法无效。如果定时器正在运行，调用`cancel()`将停止定时器，并且目标函数不会被调用。

2.3.3 简单应用

1. Timer的简单应用

```
import threading
import time


def get_time():
    # 获取当前时间的时间戳
    current_time = time.time()
    # 将时间戳格式化为年月日时分秒的格式
    formatted_time = time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(current_time))
    print(formatted_time)
```

华清远见云宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
if __name__ == '__main__':
    now_time = time.time()
    # 将时间戳格式化为年月日时分秒的格式
    formatted_time = time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(now_time))
    print(formatted_time)

    # 创建一个Timer，5秒后执行hello函数
    timer = threading.Timer(5, get_time)
    timer.start()

    timer.cancel()

    # 在主线程中做一些其他事情
    print("Do some other things...")

    # 等待定时器完成（如果需要的话）
    timer.join()
```

2. Timer的循环应用

```
import threading
import time


def get_time():
    # 获取当前时间的时间戳
    current_time = time.time()
    # 将时间戳格式化为年月日时分秒的格式
    formatted_time = time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(current_time))
    print(formatted_time)
    timer = threading.Timer(5, get_time)
    timer.start()


# 创建一个Timer，5秒后执行hello函数
timer = threading.Timer(5, get_time)
timer.start()

# 在主线程中做一些其他事情
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
print("Do some other things...")

# 等待定时器完成（如果需要的话）
timer.join()
```

3. 线程间通信

3.1 消息队列

在Python中，`queue`模块提供了一个适用于多线程环境的队列实现，这里介绍三种消息队列，分别是**Queue**、**LifoQueue**和**PriorityQueue**。

3.1.1 Queue

最常用的消息队列，遵循先进先出的原则，使用`queue.Queue`创建。

```
queue.Queue(maxsize=0)
```

- `maxsize`：队列的最大尺寸。如果设置为小于或等于0的数，则队列的尺寸是无限的。

以下是`queue.Queue`类的方法：

- `Queue.qsize()`：返回队列中当前有几条消息。
- `Queue.empty()`：如果队列为空，返回`True`，否则返回`False`。
- `Queue.full()`：如果队列已满（达到最大尺寸），返回`True`，否则返回`False`。
- `Queue.put(item, block=True, timeout=None)`：将`item`放入队列。如果`block`是`True`且`timeout`是`None`（默认），则在必要时阻塞至有空闲的插槽。如果`timeout`是正数，将最多阻塞`timeout`秒，如果在这段时间内没有可用的插槽，将引发`queue.Full`异常。
- `Queue.put_nowait(item)`：相当于`Queue.put(item, block=False)`。如果队列已满，立即引发`queue.Full`异常。
- `Queue.get(block=True, timeout=None)`：从队列中移除并返回一个元素。如果`block`是`True`且`timeout`是`None`（默认），则在必要时阻塞至队列中有项目可用。如果`timeout`是正数，将最多阻塞`timeout`秒，如果在这段时间内没有项目可用，将引发`queue.Empty`异常。
- `Queue.get_nowait()`：相当于`Queue.get(block=False)`。如果队列为空，立即引发`queue.Empty`异常。

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- `Queue.task_done()`：指示之前入队的一个任务已经完成。由队列的消费者线程使用。每个 `Queue.get()` 调用之后，需要调用 `Queue.task_done()` 告诉队列该任务处理完成。
- `Queue.join()`：阻塞调用线程，直到队列中的所有项目都被处理完（即队列中每个项目都有一个对应的 `Queue.task_done()` 调用）。

```
from threading import Thread
from queue import Queue
import time

def process1(thread_queue):
    print('准备接收数据')
    received_data = thread_queue.get()  # 接收数据
    print('接收到的数据为: ', received_data)


if __name__ == '__main__':
    thread_queue = Queue(5)
    t1 = Thread(target=process1, args=(thread_queue,))
    t1.start()
    time.sleep(2)
    thread_queue.put('hello')  # 发送数据
    t1.join()  # 等待线程结束
```

使用线程完成的生产者-消费者模式：

```
from threading import Thread
from queue import Queue
import time

# 生产者函数
def producer(queue):
    for i in range(5):
        queue.put(f"Product {i}")
        print(f"Produced {i}")
        time.sleep(1)

# 消费者函数
def consumer(queue):
    while True:
        product = queue.get()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        if product is None:
            break  # 接收到结束信号，退出循环
        print(f"Consumed {product}")
        time.sleep(1)

if __name__ == '__main__':
    queue = Queue(5)
    p = Thread(target=producer, args=(queue,))
    c = Thread(target=consumer, args=(queue,))

    p.start()
    c.start()

    p.join()  # 等待生产者线程结束
    queue.put(None)  # 发送结束信号给消费者
    c.join()  # 等待消费者线程结束
```

3.1.2 LifoQueue

用于实现后进先出（Last In, First Out）的队列。与标准的 `queue.Queue`（先进先出，FIFO）不同，`LifoQueue` 允许最后被放入队列的元素最先被取出，使用 `queue.LifoQueue` 创建。

```
queue.LifoQueue(maxsize=0)
```

- `maxsize`：队列的最大尺寸。如果设置为小于或等于0的数，则队列的尺寸是无限的。

常用方法：

- `LifoQueue.put(item, block=True, timeout=None)`：将 `item` 放入队列。如果 `block` 是 `True` 且 `timeout` 是 `None`（默认），则在必要时阻塞至有空闲的插槽。如果 `timeout` 是正数，将最多阻塞 `timeout` 秒，如果在这段时间内没有可用的插槽，将引发完全异常。
- `LifoQueue.put_nowait(item)`：相当于 `LifoQueue.put(item, block=False)`。如果队列已满，立即引发完全异常。
- `LifoQueue.get(block=True, timeout=None)`：从队列中移除并返回一个元素。如果 `block` 是 `True` 且 `timeout` 是 `None`（默认），则在必要时阻塞至队列中有项目可用。如果 `timeout` 是正数，将最多阻塞 `timeout` 秒，如果在这段时间内没有项目可用，将引发完全异常。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- `LifoQueue.get_nowait()`：相当于 `LifoQueue.get(block=False)`。如果队列为空，立即引发完全异常。
- `LifoQueue.qsize()`：返回队列中的项目数量。
- `LifoQueue.empty()`：如果队列为空，返回 `True`，否则返回 `False`。
- `LifoQueue.full()`：如果队列已满（达到最大尺寸），返回 `True`，否则返回 `False`。

```
import queue

# 创建一个 LifoQueue
lifo_queue = queue.LifoQueue(5)

# 向 LifoQueue 中放入元素
lifo_queue.put('First')
lifo_queue.put('Second')
lifo_queue.put('Third')

# 从 LifoQueue 中取出元素
print(lifo_queue.get())
print(lifo_queue.get())
print(lifo_queue.get())
```

3.1.3 PriorityQueue

PriorityQueue用于实现优先级队列，在 `PriorityQueue` 中，元素被赋予一个优先级值，并且元素会按照优先级顺序被取出。在Python中，使用 `queue.PriorityQueue` 来创建。

```
queue.PriorityQueue(maxsize=0)
```

- `maxsize`：队列的最大尺寸。如果设置为小于或等于0的数，则队列的尺寸是无限的。

常用方法：

- `PriorityQueue.put((priority, item), block=True, timeout=None)`：将 `item` 放入队列，并为其指定一个优先级 `priority`。如果 `block` 是 `True` 且 `timeout` 是 `None`（默认），则在必要时阻塞至有空闲的插槽。如果 `timeout` 是正数，将最多阻塞 `timeout` 秒，如果在这段时间内没有可用的插槽，将引发完全异常。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- `PriorityQueue.put_nowait((item, priority)`：相当于 `PriorityQueue.put((item, priority), block=False)`。如果队列已满，立即引发完全异常。
- `PriorityQueue.get(block=True, timeout=None)`：从队列中移除并返回一个元素。如果 `block` 是 `True` 且 `timeout` 是 `None`（默认），则在必要时阻塞至队列中有项目可用。如果 `timeout` 是正数，将最多阻塞 `timeout` 秒，如果在这段时间内没有项目可用，将引发完全异常。
- `PriorityQueue.get_nowait()`：相当于 `PriorityQueue.get(block=False)`。如果队列为空，立即引发完全异常。
- `PriorityQueue.qsize()`：返回队列中的项目数量。
- `PriorityQueue.empty()`：如果队列为空，返回 `True`，否则返回 `False`。
- `PriorityQueue.full()`：如果队列已满（达到最大尺寸），返回 `True`，否则返回 `False`。

```
import queue

# 创建一个 PriorityQueue
priority_queue = queue.PriorityQueue(5)

# 向 PriorityQueue 中放入元素
priority_queue.put((2, 'Task1'))
priority_queue.put((0, 'Task2'))
priority_queue.put((1, 'Task3'))

# 从 PriorityQueue 中取出元素
print(priority_queue.get())
print(priority_queue.get())
print(priority_queue.get())
```

4. 线程同步

在Python中，线程同步指的是一系列用于控制多个线程访问共享资源的方法和规则，以避免数据不一致或竞争条件（Race Condition）的问题。由于线程是操作系统调度的基本单元，它们可能会同时操作同一份数据，这可能会导致数据错误或难以预测的结果。以下是几种常见的线程同步机制：

1. **锁（Locks）**：锁是最基本的同步机制。在Python中，可以通过`threading`模块的`Lock`类来实现。锁可以确保同一时间只有一个线程能够访问共享资源。线程在访问资源前必须获取锁，访问结束后释放锁。

华清远见云计算实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

2. **信号量（Semaphores）**：信号量是一个更高级的同步机制，它维护了一个计数器，线程可以增加或减少这个计数器。如果计数器为零，则线程会阻塞，直到其他线程增加计数器。
3. **事件（Events）**：事件是一种线程之间的通信机制。一个线程可以设置事件，而其他线程可以等待该事件的发生。这可以用来通知一个或多个线程某个条件已经满足。
4. **条件变量（Condition Variables）**：条件变量通常与互斥锁一起使用，它允许线程在某个条件不满足时挂起（等待），直到另一个线程通知条件已经满足。
5. **屏障（Barriers）**：屏障是一种同步机制，允许多个线程在某个点上同步，直到所有线程都到达屏障点后，才能继续执行。

4.1 锁

与进程中类似，Python中的线程锁也有两个，一个是普通的锁Lock，另一个是可重入锁RLock。

4.1.1 Lock

要使用 `Lock`，首先需要导入 `threading` 模块，并创建一个 `Lock` 对象：

```
import threading

lock = threading.Lock()
```

Lock的操作：

- **获取锁**：使用 `lock.acquire()` 方法可以获取锁。如果锁已经被其他线程持有，则调用此方法的线程将被阻塞，直到锁被释放。
- **释放锁**：使用 `lock.release()` 方法可以释放锁。释放锁后，其他线程可以获取该锁。

在使用时，要注意锁的释放，否则会造成死锁，建议直接使用with语句对锁进行资源管理。

```
from threading import Thread, Lock
from queue import Queue
import time
import sys


def producer(queue):
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    while True:
        queue.put('hello')
        time.sleep(0.3)


def consumer(queue, lock, consumer_id):
    while True:
        with lock:
            if not queue.empty():
                res = queue.get()
                print(f'consumer{consumer_id}: {res}')
                sys.stdout.flush()


if __name__ == '__main__':
    queue = Queue(10)  # 设置队列的大小
    lock = Lock()

    producer_thread = Thread(target=producer, args=(queue,))
    producer_thread.start()

    consumers = []
    for i in range(1, 6):  # 创建5个消费者线程
        consumer_thread = Thread(target=consumer, args=(queue,
lock, i))
        consumer_thread.start()
        consumers.append(consumer_thread)

    # 等待生产者线程结束
    producer_thread.join()

    # 等待所有消费者线程结束
    for consumer_thread in consumers:
        consumer_thread.join()
```

4.1.2 RLock

要使用 `RLock`，首先需要导入 `threading` 模块，并创建一个 `RLock` 对象：

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
import threading

rlock = threading.RLock()
```

RLock的操作：

- **获取锁**：使用 `rlock.acquire()` 方法可以获取锁。如果锁已经被其他线程持有，则调用此方法的线程将被阻塞，直到锁被释放。如果锁已经被当前线程持有，则持有计数增加，方法立即返回。
- **释放锁**：使用 `rlock.release()` 方法可以释放锁。这会导致持有计数减少。如果计数降到零，锁被释放，其他线程可以获取。

在使用时不会造成死锁，但也要尽量去释放锁，否则只会剩下单独的线程。

```
from threading import Thread, RLock
from queue import Queue
import time
import sys

def producer(queue):
    while True:
        queue.put('hello')
        time.sleep(0.5)

def consumer(queue, lock, consumer_id):
    while True:
        with lock:
            if not queue.empty():
                res = queue.get()
                print(f'consumer{consumer_id}: {res}')
                sys.stdout.flush()

if __name__ == '__main__':
    queue = Queue(10)  # 设置队列的大小
    lock = RLock()

    producer_thread = Thread(target=producer, args=(queue,))
    producer_thread.start()

    consumers = []
    for i in range(1, 6):  # 创建5个消费者线程
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        consumer_thread = Thread(target=consumer, args=(queue, lock, i))
        consumer_thread.start()
        consumers.append(consumer_thread)

    # 注意：由于生产者和消费者线程被设计为无限循环，这里没有调用 join()。
```

4.2 信号量

在Python的`threading`模块中，`Semaphore`（信号量）是一个同步原语，用于控制对共享资源的访问数量。信号量维护一个内部计数器，这个计数器可以通过两个基本方法来控制：`acquire()`（获取）和`release()`（释放）。

- **计数信号量**：信号量是一个整数计数器，用于表示可用的资源数量。每当线程想要访问共享资源时，它必须先获取信号量。如果信号量的计数大于零，则线程可以减少计数（获取信号量），并继续执行。如果计数为零，则线程将被阻塞，直到其他线程释放信号量。
- **资源限制**：信号量通常用于限制对共享资源的并发访问数量。

创建 Semaphore 对象：

```
import threading

semaphore = threading.Semaphore(value)
```

value：表示最大的并发访问数量。

获取和释放信号量

- **获取信号量**：调用`semaphore.acquire()`方法来获取信号量。如果计数器大于零，则计数器减一，方法立即返回。如果计数器为零，调用线程将被阻塞，直到其他线程释放信号量。
- **释放信号量**：调用`semaphore.release()`方法来释放信号量。这会增加计数器的值，并唤醒等待获取信号量的线程（如果有）。

```
from threading import Thread, Semaphore
from queue import Queue
import time
import sys


def producer(queue):
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    while True:
        queue.put('hello')
        time.sleep(0.3)


def consumer(queue, semaphore, consumer_id):
    while True:
        with semaphore:
            if not queue.empty():
                res = queue.get()
                print(f'consumer{consumer_id}: {res}')
                sys.stdout.flush()



if __name__ == '__main__':
    queue = Queue(10)  # 设置队列的大小
    semaphore = Semaphore(1)  # 允许一个线程同时访问资源

    producer_thread = Thread(target=producer, args=(queue,))
    producer_thread.start()

    consumers = []
    for i in range(1, 6):  # 创建5个消费者线程
        consumer_thread = Thread(target=consumer, args=(queue,
semaphore, i))
        consumer_thread.start()
        consumers.append(consumer_thread)
```

华清远见元宇宙实验中心 yyzlab.com.cn

4.3 事件

在Python的`threading`模块中，`Event`是一个线程同步机制，它允许一个线程向一个或多个其他线程发送信号，而无需使用共享数据或锁。事件对象管理一个内部标志，线程可以使用这个标志来协调它们的工作。

创建 Event 对象

要使用`Event`，首先需要导入`threading`模块，并创建一个`Event`对象：

```
import threading

event = threading.Event()
```

华清远见 HQYJ.COM

Event的方法：

- `set()`：将事件的内部标志设置为`True`。所有等待这个事件的线程将被唤醒。
- `clear()`：将事件的内部标志设置为`False`。
- `is_set()`：返回事件的内部标志的当前值（`True`或`False`）。
- `wait([timeout])`：阻塞线程，直到事件的内部标志变为`True`。如果提供了`timeout`参数，线程将在超时后返回，不管事件是否被设置。

事件的使用分为两种情况：

1. 事件只被唤醒一次，然后开始进行自己的工作。

```
from threading import Event, Thread
import time


def producer(start_event, ):
    start_event.set()  # 设置事件以通知消费者可以开始消费
    while True:
        time.sleep(1)


def consumer1(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(1)


def consumer2(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(2)


def consumer3(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(3)


def consumer4(start_event, ):
    start_event.wait()  # 等待事件被设置
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    while True:
        print(4)


def consumer5(start_event, ):
    start_event.wait()  # 等待事件被设置
    while True:
        print(5)


if __name__ == '__main__':
    start_event = Event()

    producer_thread = Thread(target=producer, args=(start_event,
))
    producer_thread.start()

    consumer1_thread = Thread(target=consumer1, args=(start_event,
))
    consumer1_thread.start()

    consumer2_thread = Thread(target=consumer2, args=(start_event,
))
    consumer2_thread.start()

    consumer3_thread = Thread(target=consumer3, args=(start_event,
))
    consumer3_thread.start()

    consumer4_thread = Thread(target=consumer4, args=(start_event,
))
    consumer4_thread.start()

    consumer5_thread = Thread(target=consumer5, args=(start_event,
))
    consumer5_thread.start()

    producer_thread.join()
    consumer1_thread.join()
    consumer2_thread.join()
    consumer3_thread.join()
    consumer4_thread.join()
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    consumer5_thread.join()
```

2. 工作的执行时刻都需要事件来控制。

```
from threading import Thread, Event
import time


def producer(start_event, ):
    while True:
        start_event.set()  # 设置事件以通知消费者可以开始消费
        time.sleep(0.1)
        start_event.clear()
        time.sleep(3)


def consumer1(start_event, ):
    while True:
        start_event.wait()  # 等待事件被设置
        print(1)


def consumer2(start_event, ):
    while True:
        start_event.wait()  # 等待事件被设置
        print(2)


def consumer3(start_event, ):
    while True:
        start_event.wait()  # 等待事件被设置
        print(3)


def consumer4(start_event, ):
    while True:
        start_event.wait()  # 等待事件被设置
        print(4)


def consumer5(start_event, ):
    while True:
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        start_event.wait()  # 等待事件被设置
        print(5)


if __name__ == '__main__':
    start_event = Event()

    producer_thread = Thread(target=producer, args=(start_event,
))
    producer_thread.start()

    consumer1_thread = Thread(target=consumer1, args=(start_event,
))
    consumer1_thread.start()

    consumer2_thread = Thread(target=consumer2, args=(start_event,
))
    consumer2_thread.start()

    consumer3_thread = Thread(target=consumer3, args=(start_event,
))
    consumer3_thread.start()

    consumer4_thread = Thread(target=consumer4, args=(start_event,
))
    consumer4_thread.start()

    consumer5_thread = Thread(target=consumer5, args=(start_event,
))
    consumer5_thread.start()

    producer_thread.join()
    consumer1_thread.join()
    consumer2_thread.join()
    consumer3_thread.join()
    consumer4_thread.join()
    consumer5_thread.join()
```

华清远见元宇宙实验中心 yyzlab.com.cn

4.4 条件变量

华清远见 HQYJ.COM

在Python的`threading`模块中，`Condition`（条件变量）是一个高级同步机制，它允许线程在某些条件满足时进行等待，并在这些条件不满足时继续执行。与多进程中的Condition一样，多线程中的Condition内部也提供了一个RLock，这意味着它提供了锁机制，并在此基础上提供了条件等待和通知的功能。

创建 Condition 对象

要使用`Condition`，首先需要导入`threading`模块，并创建一个`Condition`对象：

```
import threading

condition = threading.Condition()
```

Condition 的方法：

- `acquire()`：获取条件变量的锁。如果锁已经被其他线程持有，则调用线程将被阻塞，直到锁被释放。
- `release()`：释放条件变量的锁。
- `wait()`：释放条件变量的锁，并将调用线程阻塞，直到其他线程调用`notify()`或`notify_all()`。
- `wait_for(predicate, timeout=None)`：释放内部锁，并使调用线程阻塞。*predicate* 应该是一个可调用对象而且它的返回值可被解释为一个布尔值，如果为True，则线程被唤醒，False就继续等待。 *timeout* 参数给出最大等待时间。
- `notify()`：唤醒一个等待的线程。如果有多于一个线程在等待，则随机选择一个唤醒。
- `notify_all()`：唤醒所有等待的线程。

4.4.1 使用默认锁

```
from threading import Thread, Condition
import time

def producer(condition):
    while True:
        condition.acquire()
        condition.notify_all()  # 通知所有等待的消费者
        condition.release()
        time.sleep(3)  # 模拟生产者的工作周期

def consumer(condition, number):
    while True:
```

华清远见宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        condition.acquire()
        print(f'{number}正在等待condition')
        condition.wait()  # 等待条件被满足
        print(f'{number}已释放condition')
        # 证明condition自带的是RLock，而不是Lock
        # condition.release()

if __name__ == '__main__':
    condition = Condition()

    producer_thread = Thread(target=producer, args=(condition,))
    producer_thread.start()

    consumers = [
        Thread(target=consumer, args=(condition, 1)),
        Thread(target=consumer, args=(condition, 2)),
        Thread(target=consumer, args=(condition, 3)),
        Thread(target=consumer, args=(condition, 4)),
        Thread(target=consumer, args=(condition, 5))
    ]

    for c in consumers:
        c.start()

    producer_thread.join()
    for c in consumers:
        c.join()
```

4.4.2 使用with管理

```
from threading import Thread, Condition
import time

def producer(condition):
    while True:
        time.sleep(3)  # 模拟生产者的工作周期
        with condition:  # 使用with语句自动管理锁
            condition.notify_all()  # 通知所有等待的消费者

def consumer(condition, number):
    while True:
```

华清远见元宇宙实验中心 hqyyzlab.com.cn

华清远见 HQYJ.COM

```
        with condition:  # 使用with语句自动管理锁
            print(f'{number}正在等待condition')
            condition.wait()  # 等待条件被满足
            print(f'{number}已释放condition')

if __name__ == '__main__':
    condition = Condition()

    producer_thread = Thread(target=producer, args=(condition,))
    producer_thread.start()

    consumers = [
        Thread(target=consumer, args=(condition, 1)),
        Thread(target=consumer, args=(condition, 2)),
        Thread(target=consumer, args=(condition, 3)),
        Thread(target=consumer, args=(condition, 4)),
        Thread(target=consumer, args=(condition, 5))
    ]

    for c in consumers:
        c.start()

    producer_thread.join()
    for c in consumers:
        c.join()
```

4.4.3 使用自定义锁

在大多数情况下，不需要额外的锁。`Condition` 对象内部的锁已经足够来管理条件变量的操作。

```
from threading import Thread, Condition, Lock
import time


def producer(condition):
    while True:
        with condition:  # 使用with语句自动管理锁
            condition.notify_all()  # 通知所有等待的消费者
        time.sleep(3)  # 模拟生产者的工作周期
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
def consumer(condition, number):
    while True:
        with condition:  # 使用with语句自动管理锁
            print(f'{number}抢到了锁，正在等待condition')
            condition.wait()  # 等待条件被满足
            print(f'condition已经触发，{number}释放了锁，')


if __name__ == '__main__':
    lock = Lock()
    condition = Condition(lock)

    producer_thread = Thread(target=producer, args=(condition,))
    producer_thread.start()

    consumers = [
        Thread(target=consumer, args=(condition, 1)),
        Thread(target=consumer, args=(condition, 2)),
        Thread(target=consumer, args=(condition, 3)),
        Thread(target=consumer, args=(condition, 4)),
        Thread(target=consumer, args=(condition, 5))
    ]

    for c in consumers:
        c.start()

    producer_thread.join()
    for c in consumers:
        c.join()
```

华清远见|元宇宙实验中心 yyzlab.com.cn

4.5 屏障

在Python的`threading`模块中，`Barrier`（屏障）是一种同步机制，用于让一组线程在某个点上同步。当所有线程都到达屏障点时，它们将继续执行；如果任何线程没有到达屏障点，则所有线程都会被阻塞，直到所有线程都到达。

创建Barrier对象

要使用`Barrier`，首先需要导入`threading`模块，并创建一个`Barrier`对象：

华清远见 HQYJ.COM

```
import threading

barrier = threading.Barrier(parties, action=None, timeout=None)
```

- `parties`：屏障点上需要等待的线程数量。
- `action`（可选）：当所有线程到达屏障点时，可以执行的一个函数。
- `timeout`（可选）：默认的超时时间，如果wait没有指定时间将使用这个时间。

Barrier的方法与属性：

- `wait(timeout)`：阻塞线程，直到屏障被释放。如果所有线程都到达屏障点，屏障将被释放，所有线程继续执行；如果任何线程没有到达，所有线程将被阻塞。如果提供了timeout，这里的timeout会优先于创建Barrier对象时提供的timeout参数。该函数会返回一个整数，取值在0~(parties-1)之间。
- `reset()`：重置Barrier为默认的初始状态。如果Barrier中仍有线程等待释放，将会引发异常。
- `abort()`：使Barrier处于破损状态，这将导致任何现有和未来对wait()方法的调用失败并引发异常。
- `parties`：冲出Barrier所需要的线程数量。
- `n_waiting`：当前时刻正在Barrier中阻塞的线程数量。
- `broken`：一个布尔值，表示Barrier是否为破损态。

```
import threading
import time
import random


def worker(barrier):
    print(f"Worker {threading.current_thread().name} is waiting
for the barrier.")
    seconds = random.randint(0, 5)
    print(f"Worker {threading.current_thread().name} is waiting
{seconds}.")
    time.sleep(seconds)
    barrier.wait()
    print(f"Worker {threading.current_thread().name} has passed
the barrier.")

if __name__ == '__main__':
    barrier = threading.Barrier(5)
```

华清远见 元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
workers = [
    threading.Thread(target=worker, args=(barrier,)),
    threading.Thread(target=worker, args=(barrier,)),
    threading.Thread(target=worker, args=(barrier,)),
    threading.Thread(target=worker, args=(barrier,)),
    threading.Thread(target=worker, args=(barrier,)),
]

for worker in workers:
    worker.start()

for worker in workers:
    worker.join()
```

5. 线程池

5.1 线程池的概念

线程池维护一个工作线程的集合，用于执行任务。当任务到达时，线程池可以选择一个空闲的线程来处理任务，而不是为每个任务都创建一个新的线程。

5.2 为什么使用线程池

- **降低资源消耗**：线程的创建和销毁开销较大，通过重用线程，可以降低这些开销。
- **提高响应速度**：不需要等待线程的创建就能立即执行任务。
- **提高线程的可管理性**：线程池可以统一管理线程的创建、销毁、数量等内容。

5.3 Python中的线程池实现

- **concurrent.futures.ThreadPoolExecutor**：Python标准库中的`concurrent.futures`模块提供了一个高层次的异步执行接口，`ThreadPoolExecutor`是其提供的线程池实现。

5.3.1 线程池的创建

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
concurrent.futures.ThreadPoolExecutor(max_workers=None,
thread_name_prefix='',
                                        initializer=None,
initargs=())
```

- `max_workers`：线程池中线程的最大数量。如果设置为`None`或未指定，为 min(32, os.cpu_count() + 4)， 这个默认值会保留至少 5 个工作线程用于 I/O 密集型任务。 对于那些释放了 GIL 的 CPU 密集型任务，它最多会使用 32 个 CPU 核心。这样能够避免在多核机器上不知不觉地使用大量资源。
- `thread_name_prefix`：线程名称的前缀，有助于调试时识别线程。
- `initializer`：一个可选的可调用对象，每个工作线程在启动时都会调用它。这可以用来执行线程的初始化操作，例如设置线程局部存储。
- `initargs`：一个元组，其中包含传递给`initializer`的可调用对象的参数。

`initializer`和`initargs`参数通常用于为每个线程设置特定的环境或上下文。

5.3.2 线程池的方法

1. **submit(fn, *args, **kwargs)**: 提交一个可调用的函数`fn`和必要的参数`args`和`kwargs`到线程池中执行。这个方法返回一个`Future`对象。
2. **map(func, *iterables, timeout=None, chunksize=1)**: 它允许你将一个函数`func` 应用于多个可迭代对象 `iterables` 中的元素，并且并行地在多个线程上执行这些函数调用。`timeout`是可选参数，用于设置阻塞等待每个任务完成的最大秒数。如果 `timeout` 被触发，将引发`concurrent.futures.TimeoutError`。`chunksize`是可选参数，用于指定每次切割几个参数给func，不过只对进程池有效果，对线程池无效。返回的结果是一个迭代器。
3. **shutdown(wait=True)**: 关闭线程池，停止接收新任务。如果`wait`参数为`True`，则等待所有已提交的任务完成。当使用with语句创建线程池时，with语句会在结束后自动调用shutdown。

5.3.3 线程池的简单应用

1. submit

```python
from concurrent.futures import ThreadPoolExecutor


# 定义一个函数，计算一个数字的平方
```

华清远见 yuanjian.com.cn

华清远见 HQYJ.COM

```python
def calculate_square(number):
    return number * number


# 数字列表
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# 使用with语句创建ThreadPoolExecutor实例
with ThreadPoolExecutor(max_workers=5) as executor:

    results = []
    for num in numbers:
        result = executor.submit(calculate_square, num)
        results.append(result)

    for res in results:
        print(res.result())
```

2. map

```python
from concurrent.futures import ThreadPoolExecutor
import time

def square(n):
    """计算给定数字的平方"""
    time.sleep(0.1)  # 模拟耗时操作
    return n * n

if __name__ == '__main__':
    with ThreadPoolExecutor(max_workers=4) as executor:  # 创建包含4个线程的线程池
        numbers = range(10)
        results = list(executor.map(square, numbers))  # 并行计算每个数字的平方
        print(results)
```

华清远见|元宇宙实验中心 yyzlab.com.cn